CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 21: SIMD, Thread-level Parallelism

Instructor: Justin Yokota

Get a 2:2:3:3 ratio of votes

20% of the votes should go here

20% of the votes should go here

30% of the votes should go here

30% of the votes should go here

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Get a 2:2:3:3 ratio of votes

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Get a 2:2:3:3 ratio of votes

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Agenda

- Why Parallelism?
- Data Level Parallelism: SIMD
- Applying SIMD to Matrix Multiplication

Why Parallelism?

- To answer this question, it's useful to take a look at modern supercomputers
- Often large warehouse-scale systems, which when used properly can handle extremely fast/large computations
  - Berkeley's Savio Cluster
  - Google/Amazon computing warehouses
- Personal computers will generally have similar structures at a smaller scale
- My computer:
  - 2 GHz clock cycle
  - 16 GB RAM
  - 64-bit Intel CPU using x64 (the 64-bit version of x86 assembly)

When poll is active, respond at **pollev.com/jy314**

Text **JY314** to **22333** once to join

What is the biggest difference between a supercomputer and my computer?

- A supercomputer has a higher clock speed
- A supercomputer has faster memory accesses
- A supercomputer has a higher transistor density
- A supercomputer can perform more complicated operations in a single clock cycle
- Nothing; a supercomputer is just a bunch of regular computers wired together

To

0

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

When poll is active, respond at **pollev.com/jy314**

Text **JY314** to **22333** once to join

What is the biggest difference between a supercomputer and my computer?

- A supercomputer has a higher clock speed
- A supercomputer has faster memory accesses
- A supercomputer has a higher transistor density
- A supercomputer can perform more complicated operations in a single clock cycle
- Nothing; a supercomputer is just a bunch of regular computers wired together

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Poll locked. Responses not accepted.

What is the biggest difference between a supercomputer and my computer?

- A supercomputer has a higher clock speed
- A supercomputer has faster memory accesses
- A supercomputer has a higher transistor density
- A supercomputer can perform more complicated operations in a single clock cycle
- Nothing; a supercomputer is just a bunch of regular computers wired together

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Higher Clock Speed?

- The speed of light is ~300,000,000 m/s
  - 1 GHz = 1/1,000,000,000 s, so 1 clock cycles per nanosecond
  - 3e8 m/s * 1s/1e9 ns = 0.3 m/ns
- Light travels 30 cm = about one foot in one nanosecond
- My clock cycle is 2 GHz, which is a clock cycle every half-nanosecond
- Conclusion: My computer's running so fast that the concept of simultaneity would break down if my CPU was larger than my hand
- A warehouse is larger than my hand[citation needed], so it would be physically impossible for a supercomputer to run at a significantly faster clock speed

Faster Memory Accesses/More Transistors?

- Not significantly different
- Magnetic-disk based memory accesses also are near physical limits
- More transistors = more heat = silicon starts to melt
- Small improvements possible, but not ~1000x better.

More complicated operations?

- Depends on the architecture
  - For RISC-V, it's counterproductive to have more complicated operations
  - For CISC architectures, this is actually feasible
- This leads to SIMD operations (today's topic)
- Still can only get ~4-8x better results than my PC, so relatively low effect.

More computers?

- This is the biggest difference.
- My computer has 12 independent CPUs that can run separate programs
- The Savio cluster has 3600 CPUs, with each CPU just as powerful as one of my CPUs
- Therefore, in order to gain any benefits from using a supercomputer, we need to know how to get many computers to work together on the same problem

Why Parallelism?

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \quad \frac{\text{Cycles}}{\text{Instruction}} \quad \frac{\text{Time}}{\text{Cycle}}$$

- Overall, our goal is to continue increasing the amount of computation that can be done per unit time.
- Recall the "Iron Law" of Processor Performance, which dictates the speed a program runs on one processor. In order to speed up our code, we need to improve one of:
- Instructions/Program
  - Either we reduce the work we do to solve the problem, or
  - We increase the amount of work we do per instruction
- Cycles/Instruction
- Time/Cycle

Toy Example: Vector Sum

0b0000 0001 = 1	0b0000 0010 = 2	0b0000 0011 = 3	0b0000 0100 = 4
0b0000 0101 = 5	0b0000 0110 = 6	0b0000 0111 = 7	0b0000 1000 = 8

0b0000 0110 = 6	0b0000 1000 = 8	0b0000 1010 = 10	0b0000 1100 = 12

- We have two 4-D vectors whose components are 8-bit numbers
- Goal: Determine the sum of the two vectors
- For the following example, inputs stored at 0(a0) and 0(a1), and output saved to 0(a2)

Toy Example: Vector Sum: Naive

0b0000 0001 = 1	0b0000 0010 = 2	0b0000 0011 = 3	0b0000 0100 = 4
0b0000 0101 = 5	0b0000 0110 = 6	0b0000 0111 = 7	0b0000 1000 = 8

0b0000 0110 = 6	0b0000 1000 = 8	0b0000 1010 = 10	0b0000 1100 = 12

```
lb t0 0(a0)
lb t1 0(a1)
add t0 t0 t1
sb t0 0(a2)
```

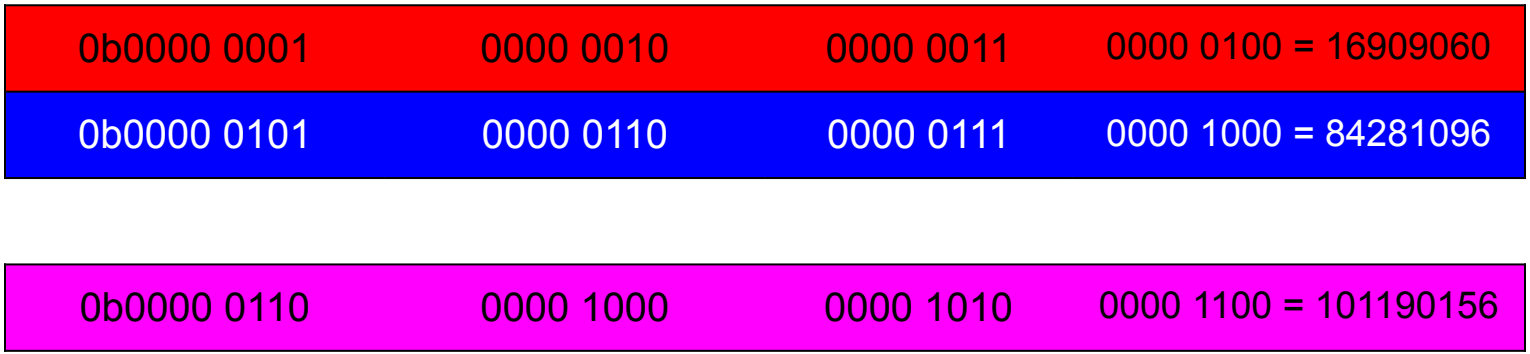
```
lb t0 1(a0)
lb t1 1(a1)
add t0 t0 t1
sb t0 1(a2)
```

```
lb t0 2(a0)
lb t1 2(a1)
add t0 t0 t1
sb t0 2(a2)
```

```
lb t0 3(a0)
lb t1 3(a1)
add t0 t0 t1
sb t0 3(a2)
```

- 16 total instructions. Can we do better?

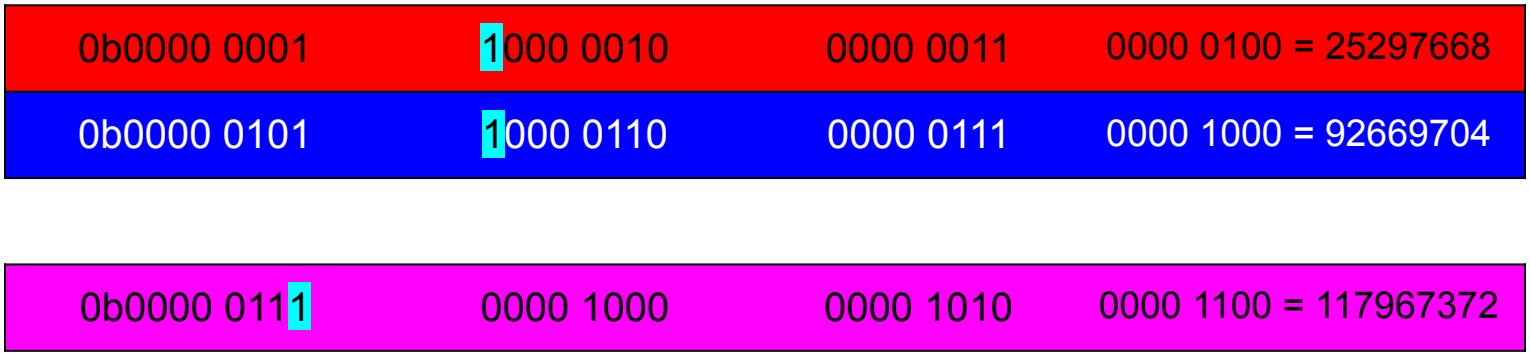
Toy Example: Vector Sum: Single Add



- Solution: If we treat these arrays as 32-bit integers, we can add with one operation, and do this in 4 instructions.

```
lw t0 0(a0)
lw t1 0(a1)
add t0 t0 t1
sw t0 0(a2)
```

Toy Example: Vector Sum: Single Add Problem



- This doesn't quite work, because overflow on one element affects other elements. So we need to create a slightly different instruction that ignores overflow every 8th bit.
- New instruction should take about as long as a single add instruction, since the circuit's similar

```
lw t0 0(a0)
lw t1 0(a1)
add t0 t0 t1
sw t0 0(a2)
```

Toy Example: Vector Sum: Vectorized Add

0b0000 0001 = 1	0b0000 0010 = 2	0b0000 0011 = 3	0b0000 0100 = 4
0b0000 0101 = 5	0b0000 0110 = 6	0b0000 0111 = 7	0b0000 1000 = 8

0b0000 0110 = 6	0b0000 1000 = 8	0b0000 1010 =10	0b0000 1100 = 12

- This doesn't quite work, because overflow on one element affects other elements. So we need to create a slightly different instruction that ignores overflow every 8th bit.
- New instruction should take about as long as a single add instruction, since the circuit's similar

```
lw t0 0(a0)
lw t1 0(a1)
vec_add t0 t0 t1
sw t0 0(a2)
```

SIMD Instructions

- Instead of doing math on one number at a time, we can instead do math on several numbers at a time, in a single clock cycle
- Known as SIMD instructions (Single-Instruction, Multiple Data) or vector instructions
- Use specialized "vector" registers which store 128, 256, or even 512 bits
- SIMD instructions act as extensions to the base instruction set, with different systems supporting different SIMD instructions.
- Generally speaking, most of the speedup comes not from doing four math operations at a time, but instead from doing a large memory load/store at a time.
    - Recall: Memory ops take 3-200x more time than arithmetic operations

SIMD Instructions

- Caveats: Each instruction needs its own circuitry, so we're limited to the set of instructions that came with the CPU
  - RISC-V doesn't have a standard vector library, so we're using x86's vector operators here.
  - In practice, this doesn't matter too much since arithmetic syntax works similarly to RISC-V
- There's still only one PC, so we can't vectorize branch or jump instructions
  - Lab discusses a way to get around that using the equivalent of slt
- Since we only have limited instructions available, we can't do different math operations to vector components, and we can only easily load consecutive blocks of memory to a vector
  - Note that most programs spend the majority of their time loading/storing instead of doing math, because loads and stores can take hundreds of cycles to resolve
- Large vectors require significant amounts of circuitry, so they are expensive to implement, and often have higher cycles/instruction than standard instructions

Intel Intrinsics

- SSE library
  - 64- and 128-bit registers: 4 32-bit integers at a time or 2 doubles at a time
- AVX library
  - 256-bit registers: 8 32-bit integers at a time or 4 doubles at a time
- AVX-2 library
  - Extension of the AVX library, with more supported instructions
- AVX-512 library
  - 512 bit registers
  - Not available on the hive machines
- AVX10
  - Successor to AVX-512, introduced in 2023 to reduce ISA complexity

Intel Intrinsics: Types

- `__m256`
  - 256 bit register for storing floats
- `__m256d`
  - 256 bit register for storing doubles
- `__m256i`
  - 256 bit register for storing 32-bit integers
- `__m128, __m128d, __m128i`
  - 128 bit registers
- Each type corresponds directly to a type of SIMD register (note that x86 has different sets of registers for floats, doubles, and integers).
- Used similarly to variables, but directly are associated with available registers, so you can't just initialize a bunch of them (or an array of them)

Intel Intrinsics: Instructions

- Generally of the format:
- `_<register size>_<instruction>_<component_type>`
- Ex. `_mm256_add_epi32` adds two 256-bit vectors, treating the vectors as arrays of 32-bit integers.
- Ex. `_mm_load_ps` loads 4 consecutive floats into a 128-bit register from the given memory address. The memory address must be aligned to a 16-byte boundary (loadu allows for nonaligned addresses, but is slower)
- More instructions: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html
- Looks like C functions, but programming with them feels more like assembly

Vector Sum (128-bit registers, 32-bit integers

0x0000 0001	0x0000 0002	0x0000 0003	0x0000 0004
0x0000 0005	0x0000 0006	0x0000 0007	0x0000 0008

0x0000 0006	0x0000 0008	0x0000 000A	0x0000 000C

```
__mm128i avec = _mm_load_si128(a);
__mm128i bvec = _mm_load_si128(b);
__mm128i sum = _mm_add_epi32(avec, bvec);
_mm_store_si128(c, sum);
```

Common mistakes when working with SIMD instructions

- Trying to directly access a 32-bit chunk of a SIMD vector (such as through typecasting)
  - Need to do an explicit load/store, since registers are different from memory
- Trying to _mm_load or _mm_store with unaligned addresses
  - Use loadu or storeu if you must, or try to get your addresses aligned
  - For mallocs, aligned_alloc gives you an aligned address
  - For local variables, you can set an attribute (example shown in slides)
- Forgetting the tail case
  - If your data isn't an array whose length is a multiple of your vector size, you need to handle the last iterations of your dataset one-by-one instead of 4 at a time.
- Using too many vectors (or creating a large array of vectors)
  - Ends up throttling your code because the compiler ends up trying to load/store SIMD vectors to the stack a bunch of times.

Applying DLP to Matrix Multiply

- Each element of the product array is the result of dot product-ing two arrays.
- Would be useful to compute one element at a time; however, we can only load data that's consecutive in memory
- Therefore, we need to start by transposing the second matrix

17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Applying DLP to Matrix Multiply

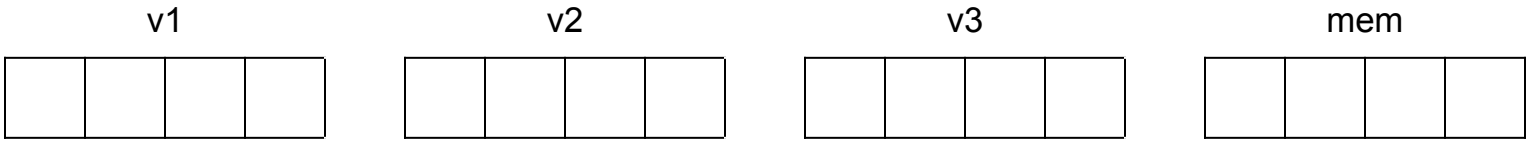
- How do we compute the dot product quickly?

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Applying DLP to Matrix Multiply

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

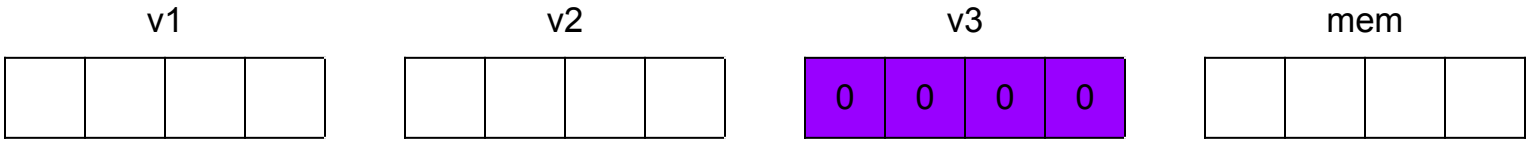


- Let's say we want to find the dot product of the two rows. How do we do this efficiently? (Assume the arrays are aligned)
- Step 1: Load 0s into v3:

```
__m128 v3 = _mm128_set1_ps(0);
```

Applying DLP to Matrix Multiply

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1



- Step 2: Load the first four numbers of each input into v1 and v2, respectively

```
__m128 v1 = _mm128_load_ps(&arr);
__m128 v2 = _mm128_load_ps(&arrtwo);
```

Applying DLP to Matrix Multiply

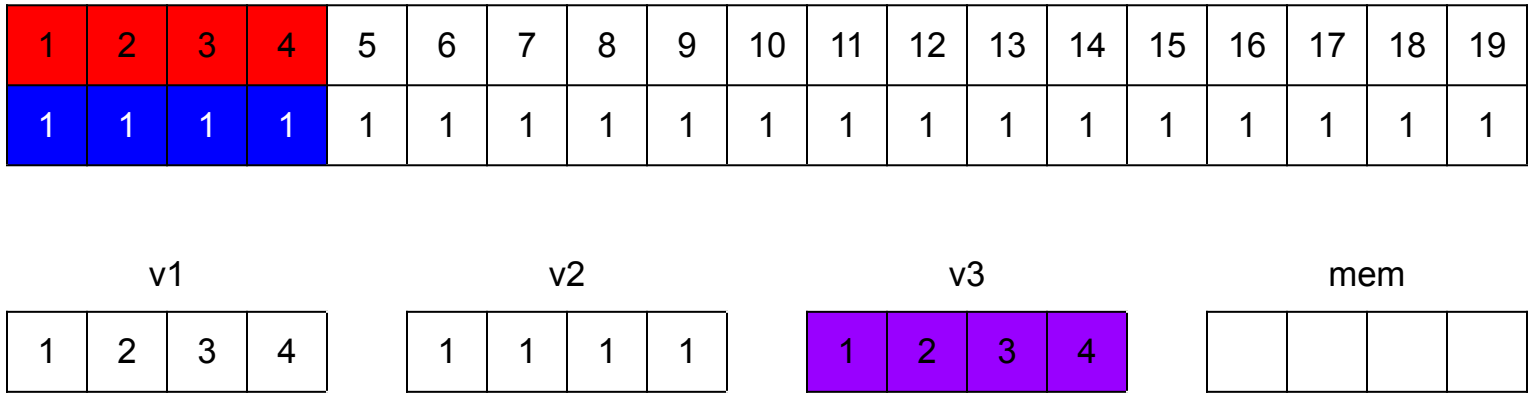
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

v1				v2				v3				mem			
1	2	3	4	1	1	1	1	0	0	0	0				

- Step 3: Multiply v1 and v2 together, and add that to v3
  - This procedure is so common, there's a single instruction to do this! (Well, for floats and doubles only)

```
v3 = _mm256_fmadd_ps(v1, v2, v3);
```

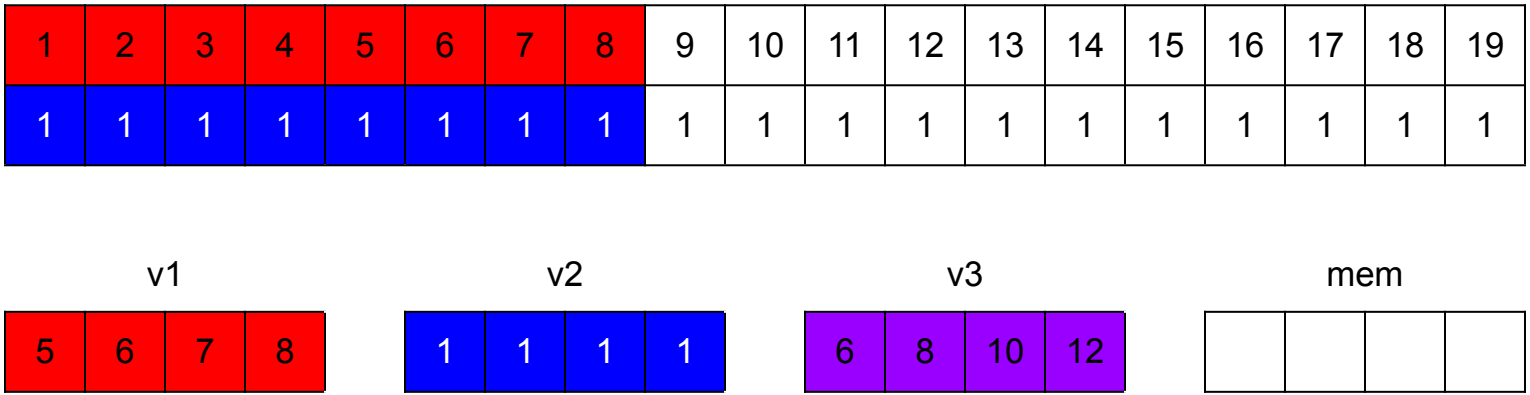
Applying DLP to Matrix Multiply



- Step 4: Repeat for the majority of the array

```
int i;
for(i = 0; i < arrlen/4*4;i+=4) {
    __m128 v1 = _mm128_load_ps(arr+i);
    __m128 v2 = _mm128_load_ps(arrtwo+i);
    v3 = _mm256_fmadd_ps(v1, v2, v3); }
```

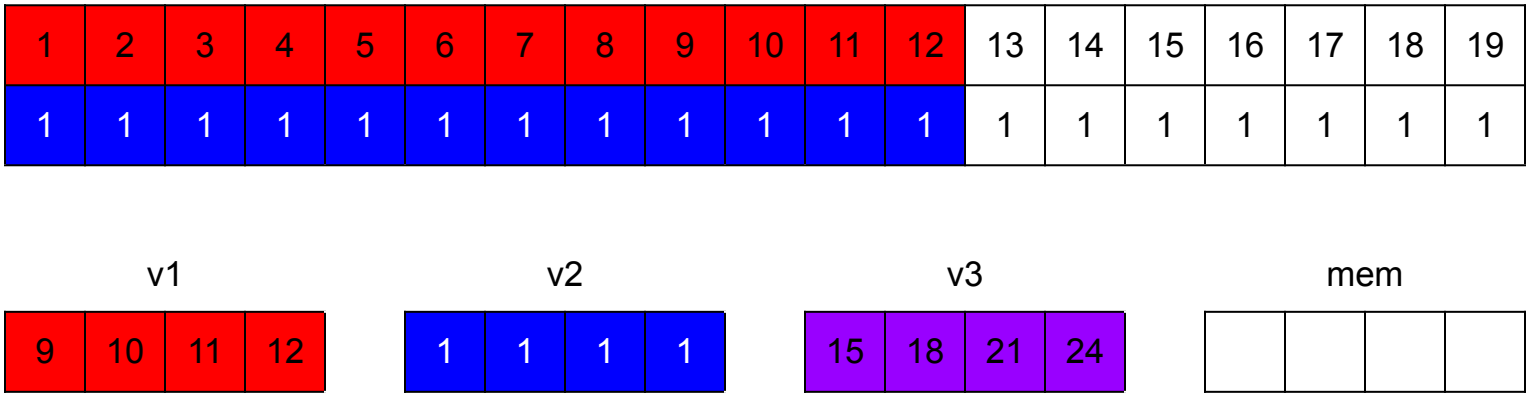
Applying DLP to Matrix Multiply



- Step 4: Repeat for the majority of the array

```
int i;
for(i = 0; i < arrlen/4*4;i+=4) {
    __m128 v1 = _mm128_load_ps(arr+i);
    __m128 v2 = _mm128_load_ps(arrtwo+i);
    v3 = _mm256_fmadd_ps(v1, v2, v3); }
```

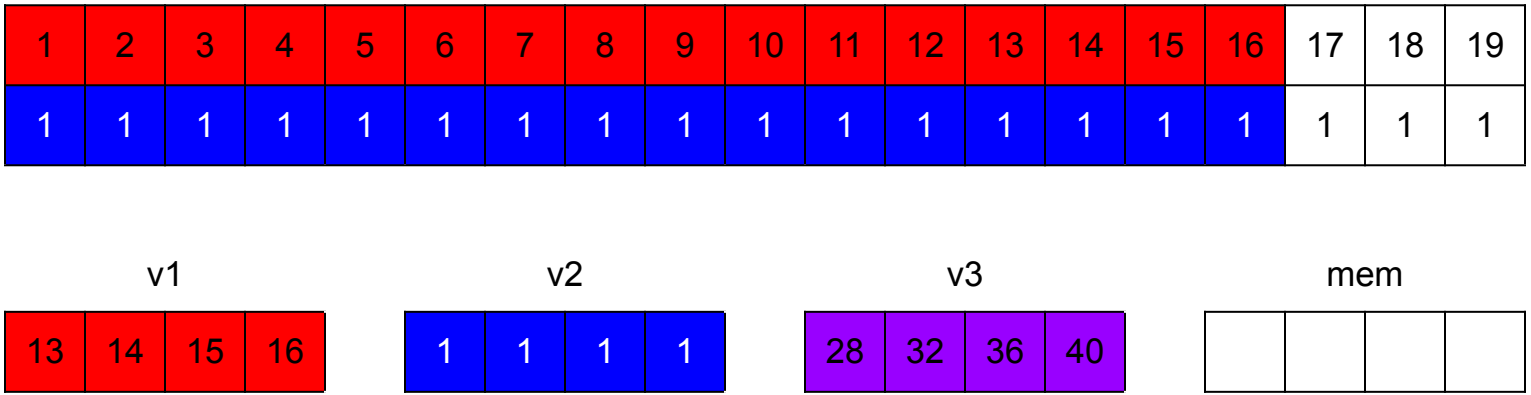
Applying DLP to Matrix Multiply



- Step 4: Repeat for the majority of the array

```
int i;
for(i = 0; i < arrlen/4*4;i+=4) {
    __m128 v1 = _mm128_load_ps(arr+i);
    __m128 v2 = _mm128_load_ps(arrtwo+i);
    v3 = _mm256_fmadd_ps(v1, v2, v3); }
```

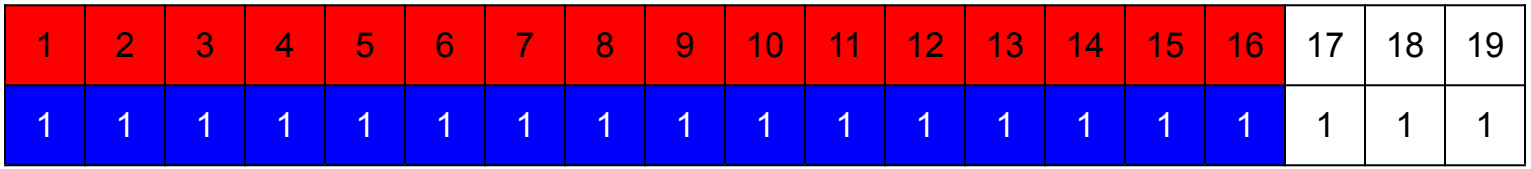
Applying DLP to Matrix Multiply

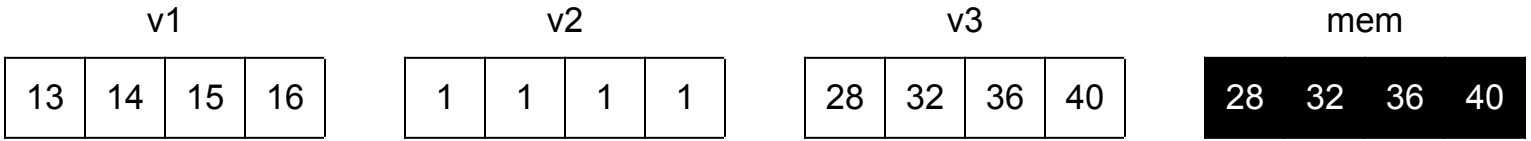


- Step 5: Store the results in memory somewhere.

```
//Force alignment
float mem[4] __attribute__ ((aligned (32)));
_mm128_store(mem, v3);
```

Applying DLP to Matrix Multiply

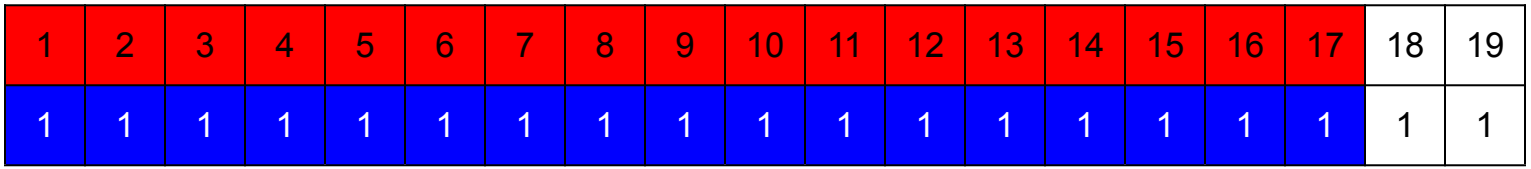


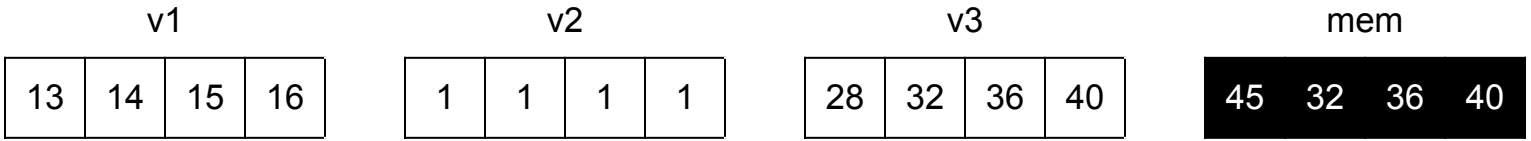


- Step 6: Resolve the tail case

```
for(;i<arrlen;i++) {
    mem[0]+=arr[i]*arrtwo[i];
}
```

Applying DLP to Matrix Multiply

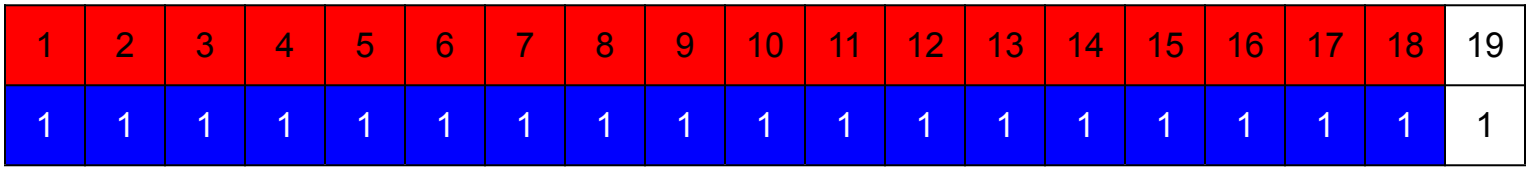


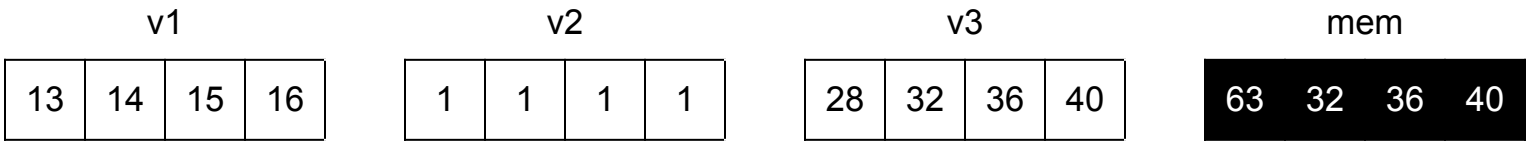


- Step 6: Resolve the tail case

```
for(;i<arrlen;i++) {
    mem[0]+=arr[i]*arrtwo[i];
}
```

Applying DLP to Matrix Multiply

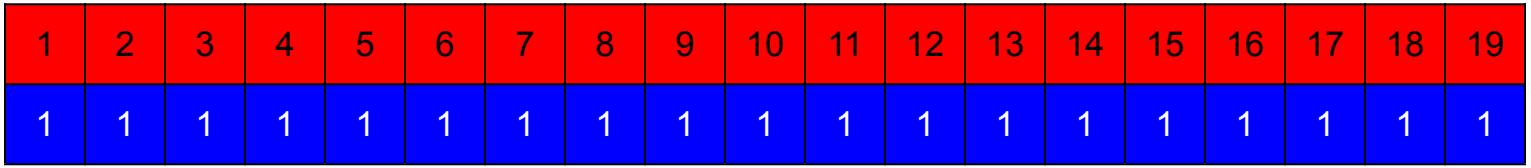


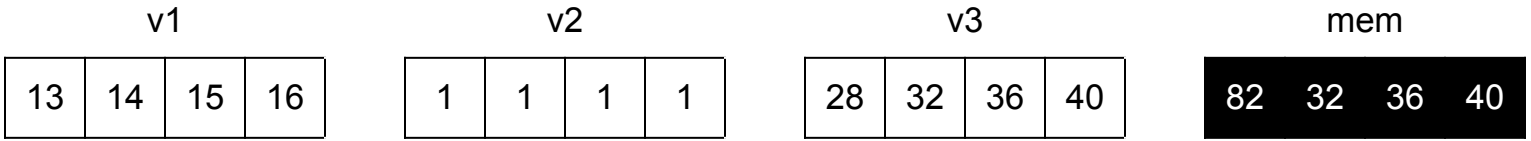


- Step 6: Resolve the tail case

```
for(;i<arrlen;i++) {
    mem[0]+=arr[i]*arrtwo[i];
}
```

Applying DLP to Matrix Multiply





- Step 7: Return the sum of mem

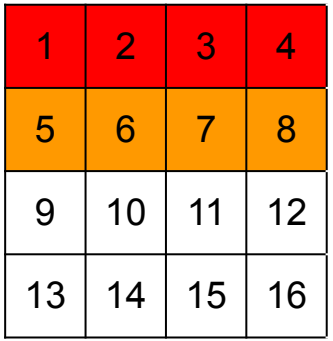
```
return mem[0]+mem[1]+mem[2]+mem[3];
```

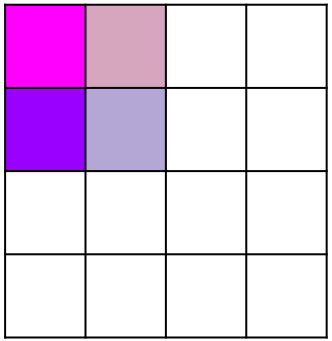
Applying DLP to Matrix Multiply


```
__m128 v3 = _mm128_set1_ps(0);
int i;
for(i = 0; i < arrlen/4*4;i+=4) {
    __m128 v1 = _mm128_load_ps(arr+i);
    __m128 v2 = _mm128_load_ps(arrtwo+i);
    v3 = _mm256_fmadd_ps(v1, v2, v3);
}
float mem[4] __attribute__ ((aligned (32)));
_mm128_store(mem, v3);
for(;i<arrlen;i++) {
    mem[0]+=arr[i]*arrtwo[i];
}
return mem[0]+mem[1]+mem[2]+mem[3];
```

Applying DLP to Matrix Multiply

- One final optimization here: right now, we load two rows to yield one value. Each row gets loaded n times.
- With enough vector registers, we can do this simultaneously to several cells at once
- This uses 8 vector registers, but computes 4 cells with 4 loads: 2x fewer loads!
- Does require an tail case for odd n





When poll is active, respond at **pollev.com/jy314**

Text **JY314** to **22333** once to join

How many times faster is this SIMD matrix multiply, compared to our original matrix multiply?

- <2x faster
- 2-10x faster
- 10-100x faster
- 100-1000x faster
- >1000x faster

To 0

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

When poll is active, respond at **pollev.com/jy314**

Text **JY314** to **22333** once to join

How many times faster is this SIMD matrix multiply, compared to our original matrix multiply?

- <2x faster
- 2-10x faster
- 10-100x faster
- 100-1000x faster
- >1000x faster

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Poll locked. Responses not accepted.

How many times faster is this SIMD matrix multiply, compared to our original matrix multiply?

- <2x faster
- 2-10x faster
- 10-100x faster
- 100-1000x faster
- >1000x faster

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Conclusion

- SIMD version still only gives a small improvement, because at this point the transpose step is taking so much of the runtime
  - Also note that int operations are faster than float operations, so the SIMD version is already at a 50% speed penalty.
- SIMD instructions are very useful when doing the same operation on a large array
- Keep in mind that loads and stores to SIMD registers take a long time, so the goal is to keep the data in registers for as long as possible; otherwise you spend most of your time in loads/stores

Agenda

- Thread-Level Parallelism
- OpenMP Syntax
- Locks and critical segments

Terminology

- A **program** is a sequence of instructions to run (such as an executable)
- A **process** is the actual execution of a program. Each process is a largely separate entity, with its own memory space.
- Each process is composed of **threads**, which are independently running instruction sequences that share most memory.
- The datapath we've discussed so far is a CPU **core**. A single core can run one thread at any given time.
- The CPU is composed of multiple cores, which thus allows multiple threads to be run simultaneously.

Terminology

- A **single-threaded program** is a program that only runs one thread
  - All the programs we've discussed so far are single-threaded
- **Multi-threaded** and **multi-process** programs are programs that use multiple threads or processes.
- The **Operating System**, or OS, is responsible for managing which threads get run on which CPUs (among other tasks)
- On most modern computers, number of active threads >> number of available cores, so most threads are idle at any given time
  - This is one of the big reasons why runtime can vary, even when running the same program
  - For Project 4, we'll run programs on dedicated systems, so you have the full resource allocation

Why Parallelism?

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \quad \frac{\text{Cycles}}{\text{Instruction}} \quad \frac{\text{Time}}{\text{Cycle}}$$

- Instructions/Program
  - Reducing the work/problem involves a lot of theory, and we'll hit theoretical limits eventually.
  - Work per instruction can be increased with SIMD, but that's at most an 8x speedup now that AVX512 is dead
- Cycles/Instruction
  - Clever ordering of instructions can help, but pipelining already optimizes this fairly well
  - Memory instructions have high cycles/instruction, so we can improve this via caches
- Time/Cycle
  - Up until recently, our limit here was manufacturing capacity, so we were able to halve this every ~2 years via Moore's Law
  - Now, we're starting to hit physical limits, so Moore's Law has slowed down quite a bit.
- Only way to move forward is to parallelize
  - Increase the number of processors that get used by a single program

Parallel Computing

- Instead of just one thread, let's write a program that runs with multiple instruction sequences simultaneously
- Why?
  - Our computer has 16 cores, so we can use them all
  - If a thread is waiting on memory accesses, might as well set up another thread to keep working
  - Supercomputers are basically normal computers with hundreds or thousands of cores, so if we want to write code for a supercomputer, we need to parallelize
- Two main choices:
  - Increase the number of threads per process: Today
  - Increase the number of processes for a program: Tomorrow (if time permits)

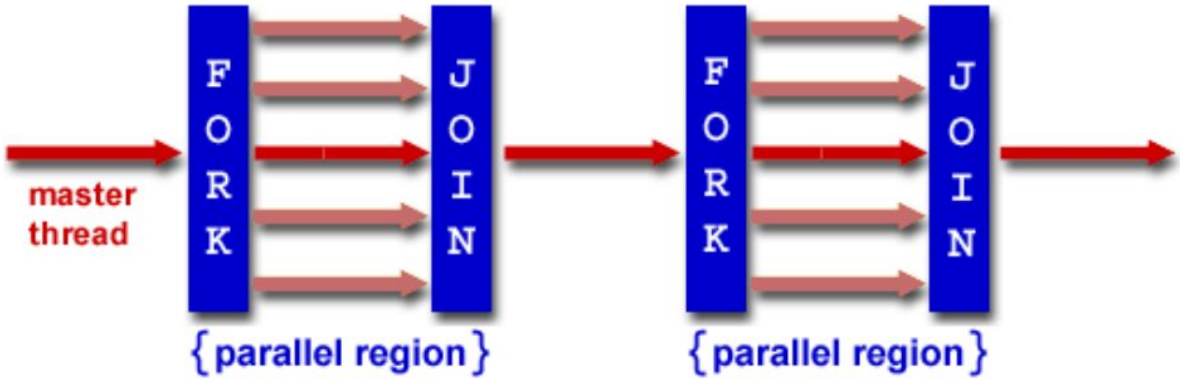
Multithreading vs Multiprocess Code

- Threads: Different instruction sequences on the same process
  - Threads on the same process share memory
  - "Easy" to communicate
  - Limited to a set of cores wired to the same memory block (1 node)
- Processes: Largely independent from each other
  - Different processes can't easily share memory
  - "Difficult" and time consuming to communicate
  - Can expand to as many cores as you have available, over as many nodes as you want
- Example: The Savio cluster is Berkeley's High Performance Computing system. It's main cluster has 112 nodes, each with 32 cores.
  - With single-threaded code: Max 1 core usable
  - With multi-threaded code: Max 32 cores = 32x speedup possible
  - With multi-process code: 112x32 cores = 3584x speedup possible
- More info: CS 267

Multithreading Framework Overview

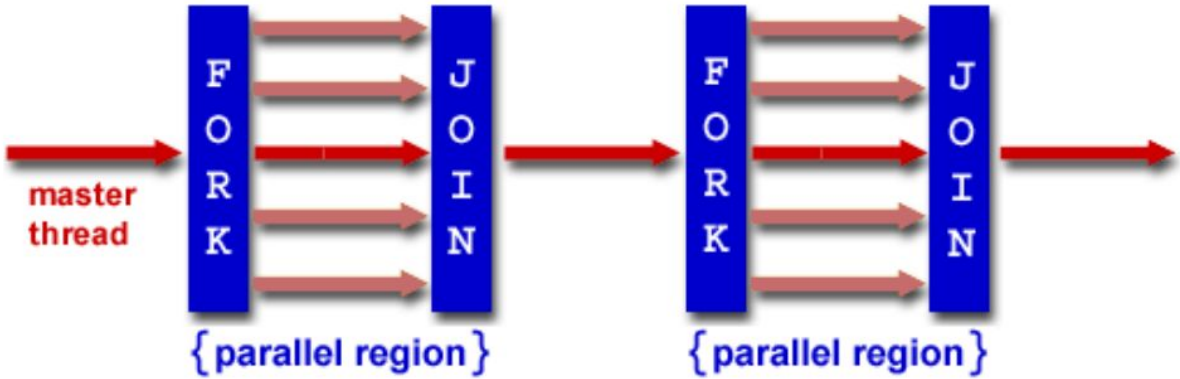
- Each thread has its own registers
- Each thread has its own PC
- Each thread has its own stack
- Each thread shares the same heap
- Communication is done through shared memory
- Threads run simultaneously, so we have no control over the order in which threads do their work

Multithreading: Fork-Join Model



- Many different multithreading models, but for this class, we'll consider the fork-join model
- Program starts serial
- Fork: Master thread creates multiple threads for a segment. Divide the work to all threads
- Join: Wait until all threads finish their work, synchronize, and terminate all but the master thread
- Fork and Join take a while (on the order of a few thousand memory operations), so goal is to minimize the number of forks/joins, and minimize the serial parts.

Multithreading: Fork-Join Model in Human Terms



- Let's say I want to make a big mural
- Step 1: I plan out things on paper, set up the wall for painting, etc.
- Step 2: Find a bunch of other people to help me paint (takes some time)
- Step 3: Assign each person some chunk of the wall to paint
- Step 4: Wait until the last person finishes painting their section
- Step 5: I do some cleanup, last minute checks, etc.

Multithreading: Scaling Efficiency

- Main problems: Minimize the nonparallelizable portion, and balance the load
- Minimize the time spent doing solo work (and overhead in finding people)
  - If the mural is small enough, it'll take more time to find people to help out, so might as well do it myself
- Strong scaling: If I double the number of people working, how much faster does the problem get (ideally close to 2x faster)?
- Weak scaling: If I double the number of people working AND double the mural size, how fast is it now (ideally close to 1x)?
- Load balancing
  - We're limited by the person who takes the most time to finish
  - Not everyone paints at the same speed, some parts of the mural might have more detail than others and therefore take longer to paint
  - Often impossible to perfectly load balance, so we have to make do with "close enough" and "statistically, everyone should have about the same amount of work"

OpenMP

- OpenMP is an extension of C used for multi-threaded code (shared memory, so no multi-node computation)
- Compiled with the additional flag "gcc -fopenmp foo.c"
  - "#include <omp.h>"
- Standardized over many languages
- Generally follows the fork-join framework
- Each thread has its own stack for private variables, but otherwise shares memory with other threads
- OpenMP code generally is written using lines like "#pragma omp <command>"

#pragma omp parallel

- In order to create a parallel section:

```
#pragma omp parallel
{
    //parallel code
}
```

- C syntax note: brackets mean "previous declaration applies to the all the lines inside me". This means that if we write only one line of code in a parallel section (or if clause, or for loop), we don't need to write brackets.
- The code in the parallel section gets run on all threads, so we need some way to distinguish threads
- In a parallel segment:
    - `omp_get_num_threads()` returns the number of threads running
    - `opm_get_thread_num()` returns a unique number from 0-num_threads per thread

Parallel Hello World


```
#include <stdio.h>
#include <omp.h>
int main () {
    int x = 0; //Shared variable
    #pragma omp parallel
    {
        int tid = omp_get_thread_num(); //Private variable
        x++;
        printf("Hello World from thread %d, x = %d\n", tid, x);
        if(tid==0) {
            printf("Number of threads = %d\n", omp_get_num_threads());
        }
    }
    printf("Done with parallel segment\n");
}
```

Shared vs Private variables

- By default, any variable declared outside the parallel segment is shared: all threads write/read from the same variable
- Any variable declared inside the parallel segment is private: Each thread has its own version of the variable.
- Can overrule this with the private and shared keywords:

```
#pragma omp parallel private(var) shared(var2)
```

- Note that heap memory is always shared, though we can have private pointers and private mallocs (if we do the malloc in the parallel segment, and free them within the parallel segment)

For loops

- Problem: You have to do some work over an array of 1 million numbers, with 4 people. How do you split the work?
- Assumptions:
  - We need to decide this before we run the code
  - Each element of the array is independent, so we can do this in any order
  - The threads are about equally fast at work, so we want to assign each of them 250k numbers

For loops

- Option 1: Do every 4
  - `for(int i = tid; i<1000000;i+=4)`
  - "Interweaving"
- Option 2: Thread 0 does 0-249999, 1 does 250000-499999, etc.
  - `for(int i = tid*250000;i<(tid+1)*250000;i++)`
  - "Blocking"
- Which one's better?
  - With standard multithreading, Option 1 actually is as slow as the serial version of this code due to cache coherency issues… More on this when we cover caching.
  - Option 2 speeds things up correctly
  - Aside: Option 1 does end up being the preferred option when dealing with GPU programming, though that's due to how GPU threads differ from normal threads.

For loops

- Option 3: Let the compiler do it for you with the `#pragma omp for` keyword.
- `#pragma omp for` must be written inside an already existing parallel segment
- If a parallel segment consists only of one for loop, we can combine the two declarations with `#pragma omp parallel for`
  - ```
    #pragma omp parallel for
        for(int i = 0; i<1000000;i++)
    ```
    ```